TRABAJO PRÁCTICO 1A

Nombre: Julieta Barahona Roth

CONSIGNA

1. Explicar qué es un socket y los diferentes tipos de sockets.
2. Cuáles son las estructuras necesarias para operar con sockets en el modelo C-S y cómo se hace para ingresar los datos requeridos. -- Explicar con un ejemplo.
3. Explicar modo bloqueante y no bloqueante en sockets y cuáles son los “sockets calls” a los cuales se pueden aplicar estos modos. -- Explicar las funciones necesarias.
4. Describir el modelo C-S aplicado a un servidor con Concurrencia Real.
5. Escribir un ejemplo en lenguaje C distinto del ya presentado.
6. Cómo se cierra una conexión C-S ?. Métodos que eviten la pérdida de informacion.
7. Probar el código del chat entre cliente y servidor. Cambiar tipo de socket y volver a probar.
8. Presentar las respuestas en archivos Jupyter Notebook o PDF.

C – S = Cliente - Servidor

Bibliografía a consultar:

-- Clases Teóricas.
-- Brian "Beej Jorgensen" Hall. Beej's Guide to Network Programming: Using Internet Sockets. 2019.
-- Douglas E. Comer, David L. Stevens. Internetworking With Tcp/Ip: Client-Server Programming and Applications. Prentice Hall. 2011.

RESPUESTAS

1. Explicar qué es un socket y los diferentes tipos de sockets.

Un socket es un punto extremo de comunicación que permite el intercambio de datos entre dos procesos (generalmente en diferentes máquinas) a través de una red.

Se usan socket () para las comunicaciones en redes la llama al sistema o “system call”, donde el mismo devuelve un descriptor de comunicaciones para usar las llamadas al sistema especializadas tales como: sen() – recv() o read()/write()

→ un descriptor de archivo es un entero asociado a un archivo abierto. Linux/unix hacen E/S escribiendo o leyendo un descriptor.

Esta definido por una **tupla de 5 elementos**:

- protocolo
- dirección IP local
- puerto local
- dirección IP remota
- puerto remoto

TIPOS DE SOCKETS
Sockets de flujo (SOCK_STREAM): definen flujos de comunicacion en dos direcciones, confiables y con conexion a TCP, son orientados a la conexion. Garantizan la entrega de datos, orden correcto y no duplicacion ya que se envian los datos en orden, llegan en el mismo orden y sin errores.
Ejemplo: HTTP, Telnet

Sockets de datagramas (SOCK_DGRAM): son sockets sin conexion, si se envia un datagrama puede llegar como no al destino. Son sin conexion porque no es necesario mantener una conexion abierta, simplemente se arma un paquete, se le coloca una cabecera IP con la informacion de destino y se envia.
Es necesario implementar un protocolo encima de UDP si se quiere asegurar que los paquetes llegen.
Ejemplos: TFTP, bootp

Sockets RAW (SOCK_RAW): es un tipo especial de socket que permite enviar y recibir paquetes sin procesar a nivel de red, saltándose las abstracciones del sistema operativo.
Se pueden crear paquetes IP, TCP, UDP o ICMP personalizados.
Para qué se usan:

- Construcción de herramientas de red (ping, traceroute, nmap).
- Implementación de protocolos personalizados.
- Monitoreo y análisis de tráfico.
- Pruebas de seguridad (ethical hacking).

2. Cuáles son las estructuras necesarias para operar con sockets en el modelo C-S y cómo se hace para ingresar los datos requeridos. -- Explicar con un ejemplo

struct sockaddr: es una estructura genérica que representa una dirección de socket.

Sirve como formato común para que funciones como bind(), connect() y accept() reciban direcciones, sin importar el tipo de protocolo usado. En ella aparece el campo:

- sa_family: indica la familia de direcciones
- sa_data: contiene la dirección y otros datos asociados

Pero en la práctica no se completa a mano, porque no es cómoda para trabajar.

struct sockaddr_in: es la estructura específica que se usa para IPv4 que tiene una forma especifica/

Su forma general es:

```
Stuct sockaddr_in {

    short sin_family

    unsignedport short sin_port

    struct in_addr sin_addr

};
```

struct in_addr: es una estructura que forma parte de sockaddr_in y representa la dirección IP.

Dentro de --> direccion.sin_addr: se guarda la IP del host con el que se quiere trabajar.

DATOS REQUERIDOS PARA LA COMUNICACION

Para que un socket pueda operar en un modelo Cliente-Servidor hay que cargar, como mínimo: la familia de direcciones, la dirección IP y el puerto. Ya que estos datos permiten identificar uno de los extremos de la comunicación.

¿Cómo se ingresan esos datos?

Los datos se cargan campo por campo dentro de --> struct sockaddr_in.

1. Familia de direcciones

```
direccion.sin_family = AF_INET;        //Esto indica que se está trabajando con IPv4.
```

2. Puerto

direccion.sin_port = htons(8080); //el puerto debe guardarse en network byte order, por eso se usa htons().

- htons() = Host to Network Short
- htonl() = Host to Network Long
- ntohs() = Network to Host Short
- ntohl() = Network to Host Long

3. Dirección IP

direccion.sin_addr.s_addr = inet_addr("127.0.0.1");

inet_aton()

4. Relleno de la estructura

memset(&(direccion.sin_zero), '\0', 8); //Esto pone en cero el campo de relleno.

Ordenacion de los bytes:

- Big Endian: primero el byte mas significativo
- Little Endian: primero el byte menos significativo

Network Byte Order (Big Endian): es un estandar que utiliza la red pero no todas las maquinas utilizan este formato ya que si necesitan convertir se utiliza host byte order.

EJEMPLO DE CARGA DE DATOS:

Struct sockaddr_in servidor;
servidor.sin_family = AF_INET;
servidor.sin_port = htons(8080);
servidor.sin_addr.s_addr = INADDR_ANY;

3. Explicar modo bloqueante y no bloqueante en sockets y cuáles son los “sockets calls” a los cuales se pueden aplicar estos modos. -- Explicar las funciones necesarias.

Modo Bloqueante: detiene la ejecucion del programa hasta que la operacion se complete, es el modo que usan por defecto los socktes.

Si se ejecuta un programa y no hay datos disponibles, el programa queda bloqueado hasta que lleguen

Las operaciones que operan en modo bloqueante son:

- accept() --> espera una conexion entrante
- recv() --> espera datos
- recvfrom() --> espera un datagrama
- read() --> lectura desde el socket

Modo No Bloqueante: no detiene la ejecucion del programa, si la ejecucion no puede completarse inmediatamente la funcion devuelve **–1** y se indica un error como **EWOULDBLOCK.**

Configurar un socket no bloqueante: *fcntl(sockfd, F_SETFL, O_NONBLOCK);*

Si se consulta constantemente un socket puede producirse un uso excesivo de CPU

4. Describir el modelo C-S aplicado a un servidor con Concurrencia Real.

El modelo Cliente-Servidor (C-S) es un esquema de comunicación en el cual el cliente inicia la comunicación enviando una solicitud y el servidor espera solicitudes y responde

En aplicaciones reales, un servidor debe ser capaz de atender múltiples clientes simultáneamente, y por eso usamos concurrencia.

Concurrencia: la concurrencia es la capacidad de un sistema de atender varias solicitudes al mismo tiempo

- de forma real (procesos/hilos paralelos)
- aparente (intercalando ejecución)

De forma real (procesos/hilos paralelos): atiende multiples clientes al mismo tiempo creando multiples procesos o hilos, uno por cliente

se usa **fork()** --> crea un proceso hijo

Funcionamiento del servidor concurrente

- El servidor crea un socket --> socket()
- Se asocia a un puerto --> bind()
- Se pone en espera --> listen()
- Acepta conexiones --> accept()

Cuando llega un cliente existen distintas formas de implementar la concurrencia:

- Mediante procesos fork() --> el proceso hijo atiende al cliente y el proceso padre sigue escuchando nuevas conexiones
- Mediante hilos threads --> se crea un hilo por cada cliente
- Multiplexacion select() --> manejar mutilples conexiones en un solo proceso

Todos los clientes se atienden en paralelo

5. Escribir un ejemplo en lenguaje C distinto del ya presentado.

Servidor:

```
tp1A — -zsh — 108×27
Servidor listo...
^C
julietabarahonaroth@Julietas-MacBook-Air tp1A % gcc servidor_nuevo.c -o servidor_nuevo
julietabarahonaroth@Julietas-MacBook-Air tp1A % gcc cleinte_nuevo.c -o cliente_nuevo
clang: error: no such file or directory: 'cleinte_nuevo.c'
clang: error: no input files
julietabarahonaroth@Julietas-MacBook-Air tp1A % gcc cliente_nuevo.c -o cliente_nuevo
julietabarahonaroth@Julietas-MacBook-Air tp1A %
julietabarahonaroth@Julietas-MacBook-Air tp1A % ./servidor_nuevo
Error en bind: Address already in use
julietabarahonaroth@Julietas-MacBook-Air tp1A %
julietabarahonaroth@Julietas-MacBook-Air tp1A %
julietabarahonaroth@Julietas-MacBook-Air tp1A % gcc servidor_nuevo.c -o servidor_nuevo
julietabarahonaroth@Julietas-MacBook-Air tp1A % gcc cliente_nuevo.c -o cliente_nuevo
julietabarahonaroth@Julietas-MacBook-Air tp1A %
julietabarahonaroth@Julietas-MacBook-Air tp1A % ./servidor_nuevo
Servidor calculadora listo en puerto 9090...
Cliente conectado.
Operacion recibida: SUMA 2 4
Operacion recibida: RESTA 9 8
Operacion recibida: MULTIPLICACION 4 2
Operacion recibida: MULT 2 4
Operacion recibida: DIV 4 2
Operacion recibida: mult 2 4
Operacion recibida: DIV 4 0
Fin de conexión
julietabarahonaroth@Julietas-MacBook-Air tp1A %
```

Cliente:

```
tp1A — -zsh — 100×24
zsh: command not found: FIN
julietabarahonaroth@Julietas-MacBook-Air tp1A %
julietabarahonaroth@Julietas-MacBook-Air tp1A %
julietabarahonaroth@Julietas-MacBook-Air tp1A % ./cliente_nuevo
Conectado al servidor calculadora.
Ingrese operaciones como: SUMA 4 5
Operaciones validas: SUMA, RESTA, MULT, DIV
Escriba FIN para terminar.
Operacion: SUMA 2 4
Resultado: 6.00
Operacion: RESTA 9 8
Resultado: 1.00
Operacion: MULTIPLICACION 4 2
Error: operacion no valida.
Operacion: MULT 2 4
Resultado: 8.00
Operacion: DIV 4 2
Resultado: 2.00
Operacion: mult 2 4
Error: operacion no valida.
Operacion: DIV 4 0
Error: division por cero.
Operacion: FIN
julietabarahonaroth@Julietas-MacBook-Air tp1A %
```

(Archivos en C adjuntados)

6. Cómo se cierra una conexión C-S ?. Métodos que eviten la pérdida de informacion

En sockets, el cierre de la conexion se realiza mediante las llamadas al sistema:

- close()
- shutdown()

Permiten finalizar la comunicacion y liberar el descriptor del socket

Función close() --> close(sockfd);

- libera el descriptor del socket
- finaliza completamente la conexión
- forma básica de cierre

Función shutdown() --> shutdown(sockfd, how);

- Permite un cierre más controlado, parcial o totalmente

Modos:
- SHUT_RD → deja de recibir
- SHUT_WR → deja de enviar
- SHUT_RDWR → ambas cosas

Si se usa close() puede haber perdida de informacion, datos en transitos o datos aun no enviados.
Para evitar pérdida de información se usa un mensaje de finalización: **FIN**

Secuencia correcta:
- Una de las partes envía "FIN"
- La otra parte lo detecta (recv() + strncmp)
- Ambas dejan de enviar datos
- Se ejecuta close()

7. Probar el código del chat entre cliente y servidor. Cambiar tipo de socket y volver a probar.

(Adjunto los codigos en c)

cliente_tcp_bash.c y servidor_tcp_bash.c :

servidor:

```
tp1A — -zsh — 80×24
dor_tcp
servidor_tcp_bash.c:1:1: warning: missing terminating '"' character [-Winvalid-p
p-token]
"Mejoras de servidor:
^
servidor_tcp_bash.c:1:1: error: expected identifier or '('
servidor_tcp_bash.c:9:44: warning: missing terminating '"' character [-Winvalid-
pp-token]
controla mejor si recv() devuelve 0 o error"
                                            ^
2 warnings and 1 error generated.
julietabarahonaroth@Julietas-MacBook-Air tp1A % gcc servidor_tcp_bash.c -o servi
dor_tcp
julietabarahonaroth@Julietas-MacBook-Air tp1A % gcc cliente_tcp_bash.c -o client
e_tcp
julietabarahonaroth@Julietas-MacBook-Air tp1A %
julietabarahonaroth@Julietas-MacBook-Air tp1A % ./servidor_tcp
Servidor esperando conexión en el puerto 8080...
Julieta: Barahona
Servidor: Hola julieta
Julieta: Como estas?
Servidor: Preparado para recibir
Julieta ha finalizado la conexión.
julietabarahonaroth@Julietas-MacBook-Air tp1A %
```

cliente:

```
tp1A — -zsh — 80×24
Last login: Wed Apr  8 19:31:04 on ttys006
julietabarahonaroth@Julietas-MacBook-Air ~ % cd Desktop/protocolosPractica/tp1A

julietabarahonaroth@Julietas-MacBook-Air tp1A % ./cliente_tcp
Ingresar nombre: Julieta
Julieta: Barahona
Servidor: Hola julieta
Julieta: Como estas?
Servidor: Preparado para recibir
Julieta: FIN
Terminando la conexión...
julietabarahonaroth@Julietas-MacBook-Air tp1A %
```

cliente_upd.c y servidor_udp.c:

servidor:

```
tp1A — -zsh — 80×24
julietabarahonaroth@Julietas-MacBook-Air tp1A % ./servidor_tcp
Servidor esperando conexión en el puerto 8080...
Julieta: Barahona
Servidor: Hola julieta
Julieta: Como estas?
Servidor: Preparado para recibir
Julieta ha finalizado la conexión.
julietabarahonaroth@Julietas-MacBook-Air tp1A %
julietabarahonaroth@Julietas-MacBook-Air tp1A %
julietabarahonaroth@Julietas-MacBook-Air tp1A % gcc servidor_udp.c -o servidor_u
dp
julietabarahonaroth@Julietas-MacBook-Air tp1A % gcc cliente_udp.c -o cliente_udp

julietabarahonaroth@Julietas-MacBook-Air tp1A % ./servidor_udp
Servidor UDP esperando mensajes en el puerto 8080...
Cliente conectado: Julieta
Julieta: Hola como estas
Servidor: Hola Julieta, bien y vos? preparado para recibir
Julieta: perfecto gracias
Servidor: OK
Julieta:
Servidor: FIN
Servidor finaliza la comunicacion.
julietabarahonaroth@Julietas-MacBook-Air tp1A %
```

cliente:

```
tp1A — -zsh — 80×24
julietabarahonaroth@Julietas-MacBook-Air tp1A % ./cliente_tcp
Ingresar nombre: Julieta
Julieta: Barahona
Servidor: Hola julieta
Julieta: Como estas?
Servidor: Preparado para recibir
Julieta: FIN
Terminando la conexión...
julietabarahonaroth@Julietas-MacBook-Air tp1A %
julietabarahonaroth@Julietas-MacBook-Air tp1A % ./cliente_udp
Ingresar nombre: Julieta
Julieta: Hola como estas
Servidor: Hola Julieta, bien y vos? preparado para recibir
Julieta: perfecto gracias

FIN
Servidor: OK
Julieta: FIN
Servidor finalizo la comunicacion.
julietabarahonaroth@Julietas-MacBook-Air tp1A % FIN
zsh: command not found: FIN
julietabarahonaroth@Julietas-MacBook-Air tp1A % FIN
zsh: command not found: FIN
julietabarahonaroth@Julietas-MacBook-Air tp1A %
```

ARCHIVOS EN C:

cliente_nuevo.c (TCP):

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 9090
#define MAX 1024

int main() {
    int sockfd;
    struct sockaddr_in server_addr;
    char buffer[MAX];

    //creo el socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == -1) {
        perror("Error al crear socket");
        exit(1);
    }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);

    if (inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr) <= 0) {
        perror("IP invalida");
        close(sockfd);
        exit(1);
    }

    if (connect(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("Error al conectar");
        close(sockfd);
        exit(1);
    }

    printf("Conectado al servidor calculadora.\n");
    printf("Ingrese operaciones como: SUMA 4 5\n");
    printf("Operaciones validas: SUMA, RESTA, MULT, DIV\n");
    printf("Escriba FIN para terminar.\n");

    while (1) {
        printf("Operacion: ");
        if (fgets(buffer, MAX, stdin) == NULL) {
            break;
        }
```

```
        send(sockfd, buffer, strlen(buffer), 0);

        if (strncmp(buffer, "FIN", 3) == 0) {
            break;
        }

        memset(buffer, 0, MAX);
        int bytes = recv(sockfd, buffer, MAX - 1, 0);

        if (bytes <= 0) {
            printf("Servidor desconectado.\n");
            break;
        }

        buffer[bytes] = '\0';
        printf("%s", buffer);
    }

    close(sockfd);
    return 0;
}
```

servidor_nuevo.c (TCP):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 9090
#define MAX 1024

//SERVIDOR CALCULADORA


int main() {
    int server_fd, client_fd;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_size;
    char buffer[MAX];
    char respuesta[MAX];
    char operacion[20];
    double a,b,resultado;

    //creo el socket
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd ==-1){
        perror("Error al crear el socket");
        exit(1);
```

```
    }

    memset(&server_addr,0,sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);


    if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        perror("Error en bind");
        close(server_fd);
        exit(1);
    }

    if (listen(server_fd, 1) < 0) {
        perror("Error en listen");
        close(server_fd);
        exit(1);
    }

    printf("Servidor calculadora listo en puerto %d...\n", PORT);

    addr_size = sizeof(client_addr);
    client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &addr_size);
    if (client_fd < 0) {
        perror("Error en accept");
        close(server_fd);
        exit(1);
    }

    printf("Cliente conectado.\n");


    while (1) {
        memset(buffer, 0, MAX);

        int bytes = recv(client_fd, buffer, MAX, 0);

        if (bytes <= 0) {
            printf("Cliente desconectado\n");
            break;
        }

        if (strncmp(buffer, "FIN", 3) == 0) {
            printf("Fin de conexión\n");
            break;
        }

        printf("Operacion recibida: %s", buffer);

        memset(operacion, 0, sizeof(operacion));
```

```c
        memset(respuesta, 0, MAX);

        if (sscanf(buffer, "%19s %lf %lf", operacion, &a, &b) == 3) {
            if (strcmp(operacion, "SUMA") == 0) {
                resultado = a + b;
                snprintf(respuesta, MAX, "Resultado: %.2f\n", resultado);
            } else if (strcmp(operacion, "RESTA") == 0) {
                resultado = a - b;
                snprintf(respuesta, MAX, "Resultado: %.2f\n", resultado);
            } else if (strcmp(operacion, "MULT") == 0) {
                resultado = a * b;
                snprintf(respuesta, MAX, "Resultado: %.2f\n", resultado);
            } else if (strcmp(operacion, "DIV") == 0) {
                if (b == 0) {
                    snprintf(respuesta, MAX, "Error: division por cero.\n");
                } else {
                    resultado = a / b;
                    snprintf(respuesta, MAX, "Resultado: %.2f\n", resultado);
                }
            } else {
                snprintf(respuesta, MAX, "Error: operacion no valida.\n");
            }
        } else {
            snprintf(respuesta, MAX, "Error: formato invalido. Use: SUMA 4 5\n");
        }

        send(client_fd, respuesta, strlen(respuesta), 0);
    }

    close(client_fd);
    close(server_fd);
    return 0;
}
```

cliente_tcp_bash.c:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define MAX 1024
#define PORT 8080

int main() {
    int sockfd;
    struct sockaddr_in server_addr;
```

```c
    char buffer[MAX];
    char name[50];

    // Crear socket
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == -1) {
        perror("Socket falló");
        exit(1);
    }

    //esctructura del socket
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);
    memset(&(server_addr.sin_zero), 0, 8);

    // Conectar al servidor
    if (connect(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr)) != 0)
{
        perror("Conexión falló");
        exit(1);
    }

    // Enviar nombre
    printf("Ingresar nombre: ");
    fgets(name, sizeof(name), stdin);
    name[strcspn(name, "\n")] = 0; // Eliminar salto de línea
    send(sockfd, name, strlen(name), 0);

    while (1) {
        // Enviar mensaje
        printf("%s: ", name);
        memset(buffer, 0, MAX);
        fgets(buffer, MAX, stdin);
        send(sockfd, buffer, strlen(buffer), 0);

        if (strncmp(buffer, "FIN", 3) == 0) {
            printf("Terminando la conexión...\n");
            break;
        }

        // Recibir respuesta
        memset(buffer, 0, MAX);
        int bytes = recv(sockfd, buffer, MAX, 0);
        if (bytes <= 0 || strncmp(buffer, "FIN", 3) == 0) {
            printf("Servidor finalizó la conexión.\n");
            break;
        }

        printf("Servidor: %s", buffer);
    }
```

```
    close(sockfd);
    return 0;
}
```

servidor_tcp_bash.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define MAX 1024    //tamaño maximo del buffer para mensajes
#define PORT 8080   //puerto que elijo, el servidor queda esperando clientes en el
puerto 8080

int main() {
    //variables principales
    int server_fd, client_fd;                       //numero entero que es como el
ID del socket abierto
    struct sockaddr_in server_addr, client_addr;    //estructura de la direccion
del socket  = IP + PUERTO + TIPO DE DIRECCION (IPv4)
    char buffer[MAX];                               //donde se guardan los mensajes
    char client_name[50];                           //donde se guarda el nombre del
cliente

    //Crear el socket
    server_fd = socket(AF_INET, SOCK_STREAM, 0);    //AF_INET = IPv4 - 0 = el
sistema elije el protocolo correcto
    if (server_fd == -1) {
        perror("Socket no pudo abrirse ... error ..."); //si es -1 = error
        exit(1);
    }

    // Configuracion de la estructura
    server_addr.sin_family = AF_INET;           //direccion IPv4
    server_addr.sin_addr.s_addr = INADDR_ANY;   //cualquier IPs locales
    server_addr.sin_port = htons(PORT);         //orden de bytes
    memset(&(server_addr.sin_zero), 0, 8);      //rellena con 0 el campo de padding
para completar la estructura

    // Asociar dirección y puerto
    //bind() le dice al sistema con que puerto y direccion trabaja el socket
    if (bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) != 0)
{
```

```
        perror("Bind falló");
        exit(1);
    }


    // Escuchar conexiones
    if (listen(server_fd, 1) != 0) {
        perror("Listen con error...");
        exit(1);
    }

    // accept() = espera a que un cliente se conecte --> bloqueante por defecto
    printf("Servidor esperando conexión en el puerto %d...\n", PORT);
    socklen_t addr_size = sizeof(client_addr);
    client_fd = accept(server_fd, (struct sockaddr*)&client_addr, &addr_size);
    if (client_fd < 0) {
        perror("Falló al aceptar conexión");
        exit(1);
    }

    //ACA MEJORAR SI RECV() FALLA O NO
    // Recibir nombre del cliente
    recv(client_fd, client_name, sizeof(client_name), 0);       //recibe los datos
desde el socket client_fd

    //hace que el servidor quede conversando con los clientes hasta que alguno
corte
    while (1) {
        // Recibir mensaje del cliente y limpiar el buffer
        memset(buffer, 0, MAX);
        int bytes = recv(client_fd, buffer, MAX, 0);
        if (bytes <= 0) {
            printf("Cliente desconectado.\n");          //error
            break;
        }

        //verificar si el cliente puso FIN
        if (strncmp(buffer, "FIN", 3) == 0) {
            printf("%s ha finalizado la conexión.\n", client_name);
//finalizo - compara los primero 3 caracteres del mensaje con FIN
            break;
        }

        // Mostrar mensaje recibido en pantalla
        printf("%s: %s", client_name, buffer);

        // Enviar respuesta
        printf("Servidor: ");
        memset(buffer, 0, MAX);
        fgets(buffer, MAX, stdin);
        send(client_fd, buffer, strlen(buffer), 0);     //manda mensaje al cliente
```

```
        //el servidor tambien puede cortar con FIN
        if (strncmp(buffer, "FIN", 3) == 0) {
            printf("Servidor finaliza la conexión.\n");
            break;
        }
    }

    // Cierre final
    close(client_fd);
    close(server_fd);
    return 0;
}
```

cliente_udp.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define MAX 1024
#define PORT 8080

int main() {
    int sockfd;
    struct sockaddr_in server_addr;
    char buffer[MAX];
    char name[50];
    socklen_t addr_size;
    int bytes;

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd == -1) {
        perror("Socket fallo");
        exit(1);
    }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);

    if (inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr) <= 0) {
        perror("Direccion invalida");
        close(sockfd);
        exit(1);
```

```c
    }

    addr_size = sizeof(server_addr);

    /* Enviar nombre */
    printf("Ingresar nombre: ");
    if (fgets(name, sizeof(name), stdin) == NULL) {
        printf("Error al leer nombre.\n");
        close(sockfd);
        exit(1);
    }
    name[strcspn(name, "\n")] = '\0';

    if (sendto(sockfd, name, strlen(name), 0,
               (struct sockaddr*)&server_addr, addr_size) < 0) {
        perror("Error al enviar nombre");
        close(sockfd);
        exit(1);
    }

    while (1) {
        printf("%s: ", name);
        memset(buffer, 0, MAX);

        if (fgets(buffer, MAX, stdin) == NULL) {
            printf("Error al leer mensaje.\n");
            break;
        }

        if (sendto(sockfd, buffer, strlen(buffer), 0,
                   (struct sockaddr*)&server_addr, addr_size) < 0) {
            perror("Error al enviar mensaje");
            break;
        }

        if (strncmp(buffer, "FIN", 3) == 0) {
            printf("Terminando la comunicacion...\n");
            break;
        }

        memset(buffer, 0, MAX);
        bytes = recvfrom(sockfd, buffer, MAX - 1, 0, NULL, NULL);

        if (bytes < 0) {
            perror("Error al recibir respuesta");
            break;
        }

        buffer[bytes] = '\0';

        if (strncmp(buffer, "FIN", 3) == 0) {
```

```
            printf("Servidor finalizo la comunicacion.\n");
            break;
        }

        printf("Servidor: %s", buffer);
    }

    close(sockfd);
    return 0;
}
```

servidor_udp.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <netinet/in.h>

#define MAX 1024
#define PORT 8080

int main() {
    int sockfd;                                     //descriptor del socket UDP
    struct sockaddr_in server_addr, client_addr;
    char buffer[MAX];
    char client_name[50];
    socklen_t addr_size;
    int bytes;

    //crea el socket UDP
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd == -1) {
        perror("Socket no pudo abrirse");
        exit(1);
    }

    //configuro la direccion del servidor
    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    //asigno al IP + direccion
    if (bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr)) != 0) {
        perror("Bind fallo");
        close(sockfd);
```

```c
        exit(1);
    }

    printf("Servidor UDP esperando mensajes en el puerto %d...\n", PORT);


    addr_size = sizeof(client_addr);        //recvfrom() necesita saber el tamaño

    //recibe el mensaje y necesita saber quien lo mando
    //controla errores
    memset(client_name, 0, sizeof(client_name));
    bytes = recvfrom(sockfd, client_name, sizeof(client_name) - 1, 0,
                     (struct sockaddr*)&client_addr, &addr_size);
    if (bytes < 0) {
        perror("Error al recibir nombre");
        close(sockfd);
        exit(1);
    }
    client_name[bytes] = '\0';

    printf("Cliente conectado: %s\n", client_name);

    while (1) {
        memset(buffer, 0, MAX);

        bytes = recvfrom(sockfd, buffer, MAX - 1, 0,
                         (struct sockaddr*)&client_addr, &addr_size);
//recibe el mensaje

        if (bytes < 0) {
            perror("Error al recibir mensaje");
            break;
        }

        buffer[bytes] = '\0';

        if (strncmp(buffer, "FIN", 3) == 0) {
            printf("%s ha finalizado la comunicacion.\n", client_name);
            break;
        }

        printf("%s: %s", client_name, buffer);

        printf("Servidor: ");
        memset(buffer, 0, MAX);

        if (fgets(buffer, MAX, stdin) == NULL) {
            printf("Error al leer desde teclado.\n");
            break;
        }
```

```
        //envia respuesta al cliente que la mando (no hay conexion previa por eso
especifica)
        if (sendto(sockfd, buffer, strlen(buffer), 0,
                   (struct sockaddr*)&client_addr, addr_size) < 0) {
            perror("Error al enviar mensaje");
            break;
        }

        if (strncmp(buffer, "FIN", 3) == 0) {
            printf("Servidor finaliza la comunicacion.\n");
            break;
        }
    }

    close(sockfd);
    return 0;
}
```